

Algoritmos greedy

TDA / Algo3

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2^{do} Cuatrimestre de 2026

Objetivo de la clase

- Reconocer problemas que admiten una solución greedy.
- Relacionar la elección local greedy con una recurrencia de programación dinámica.
- Aprender a demostrar correctitud mediante argumentos de intercambio.
- Identificar problemas donde una elección greedy no funciona.

Problema de selección de actividades

Problema

Tenemos actividades $a_1, \dots, a_n$. Cada actividad a_i tiene un tiempo de inicio s_i y un tiempo de finalización f_i . Dos actividades son compatibles si sus intervalos no se superponen. Queremos seleccionar la mayor cantidad posible de actividades compatibles.

Intervalos abiertos

La actividad a_i ocupa el intervalo $[s_i, f_i)$. Las actividades que terminan justo cuando otra empieza son compatibles.

Problema de selección de actividades

Problema

Tenemos actividades $a_1, \dots, a_n$. Cada actividad a_i tiene un tiempo de inicio s_i y un tiempo de finalización f_i . Dos actividades son compatibles si sus intervalos no se superponen. Queremos seleccionar la mayor cantidad posible de actividades compatibles.

Intervalos abiertos

La actividad a_i ocupa el intervalo $[s_i, f_i)$. Las actividades que terminan justo cuando otra empieza son compatibles.

Consigna: tómense unos minutos para proponer una función recursiva que resuelva el problema por programación dinámica.

Una primera recurrencia

Sea **SA**(**acts**, **comps**) el máximo número de actividades en la lista *acts* compatibles entre ellas y con *comps*.

$$SA(acts, comps) = \begin{cases} 0, & |acts| = 0, \\ \max \{ 1 + SA(comps, acts[0]), SA(acts[1 :]) \}, & |acts| > 0. \end{cases}$$

- $1 + SA(comps, acts[0])$: elegimos a_0 .
- $SA(acts[1 :])$: no elegimos a_0 .

Una recurrencia por programación dinámica

Podemos tener parámetros enteros en lugar de pasar las actividades y verificar en $O(1)$ las compatibilidades. Tomaremos de fin a inicio ahora.

Una recurrencia por programación dinámica

Podemos tener parámetros enteros en lugar de pasar las actividades y verificar en $O(1)$ las compatibilidades. Tomaremos de fin a inicio ahora.

Sea $\mathbf{p}(i)$ la última actividad anterior en la lista compatible con i ; si no existe, $p(i) = -1$.

Sea $\mathbf{SA}(i)$ el máximo número de actividades compatibles entre $a_0, \dots, a_i$. $SA(n-1)$ resuelve nuestro problema.

Una recurrencia por programación dinámica

Podemos tener parámetros enteros en lugar de pasar las actividades y verificar en $O(1)$ las compatibilidades. Tomaremos de fin a inicio ahora.

Sea $\mathbf{p(i)}$ la última actividad anterior en la lista compatible con i ; si no existe, $p(i) = -1$.

Sea $\mathbf{SA(i)}$ el máximo número de actividades compatibles entre $a_0, \dots, a_i$. $SA(n-1)$ resuelve nuestro problema.

$$SA(i) = \begin{cases} 0, & i = -1, \\ \max \{ 1 + SA(p(i)), SA(i-1) \}, & i \geq 0. \end{cases}$$

- $1 + SA(p(i))$: elegimos a_i .
- $SA(i-1)$: no elegimos a_i .

La recurrencia depende del orden

Ojo con el orden

En el orden listado, consideremos:

$$a_1 = [4, 5), \quad a_2 = [1, 2), \quad a_3 = [2, 3).$$

Para a_2 , las actividades compatibles son a_1 y a_3 , pero $p(2) = 1$, se pierde a_3 como opción

La recurrencia depende del orden

Ojo con el orden

En el orden listado, consideremos:

$$a_1 = [4, 5), \quad a_2 = [1, 2), \quad a_3 = [2, 3).$$

Para a_2 , las actividades compatibles son a_1 y a_3 , pero $p(2) = 1$, se pierde a_3 como opción

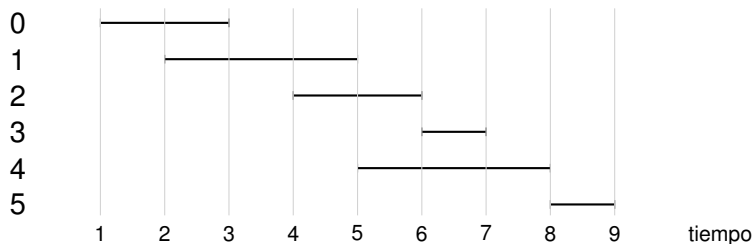
¿Se puede solucionar? Sí: ordenar por finalización antes de aplicar la recurrencia.

Ejemplo: actividades ordenadas por finalización

Consideremos las actividades:

i	0	1	2	3	4	5
$[s_i, f_i)$	$[1, 3)$	$[2, 5)$	$[4, 6)$	$[6, 7)$	$[5, 8)$	$[8, 9)$

En este orden, $p(0), \dots, p(5) = (-1, -1, 0, 2, 1, 4)$.

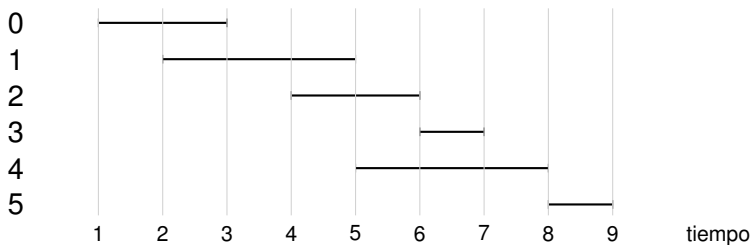


Ejemplo: actividades ordenadas por finalización

Consideremos las actividades:

i	0	1	2	3	4	5
$[s_i, f_i)$	$[1, 3)$	$[2, 5)$	$[4, 6)$	$[6, 7)$	$[5, 8)$	$[8, 9)$

En este orden, $p(0), \dots, p(5) = (-1, -1, 0, 2, 1, 4)$.



Consigna: evalúen la recurrencia $SA(5)$ y analicen qué rama conduce al óptimo.

¿La misma rama gana siempre?

En el ejemplo, la rama que elige la actividad compatible que termina antes aparece en una solución óptima.

¿La misma rama gana siempre?

En el ejemplo, la rama que elige la actividad compatible que termina antes aparece en una solución óptima.

¿Pasará siempre?

¿La misma rama gana siempre?

En el ejemplo, la rama que elige la actividad compatible que termina antes aparece en una solución óptima.

¿Pasará siempre?

Pista para discutir

Si una solución óptima empieza con una actividad que termina más tarde, ¿qué pasa si la reemplazamos por la actividad que termina antes?

Un algoritmo greedy

La elección local será tomar, entre las actividades disponibles, la que termina primero. Luego descartamos las que se superponen y repetimos.

```
1: function SELECCIONARACTIVIDADES( $A$ )
2:   ordenar  $A$  por tiempo de finalización creciente
3:    $S \leftarrow \emptyset$ 
4:    $t \leftarrow -\infty$ 
5:   for all  $a \in A$ , en el orden obtenido do
6:     if  $s_a \geq t$  then
7:        $S \leftarrow S \cup \{a\}$ 
8:        $t \leftarrow f_a$ 
9:   return  $S$ 
```

▷ *actividad compatible*

Un algoritmo greedy

La elección local será tomar, entre las actividades disponibles, la que termina primero. Luego descartamos las que se superponen y repetimos.

```
1: function SELECCIONARACTIVIDADES( $A$ )
2:   ordenar  $A$  por tiempo de finalización creciente
3:    $S \leftarrow \emptyset$ 
4:    $t \leftarrow -\infty$ 
5:   for all  $a \in A$ , en el orden obtenido do
6:     if  $s_a \geq t$  then
7:        $S \leftarrow S \cup \{a\}$ 
8:        $t \leftarrow f_a$ 
9:   return  $S$ 
```

▷ *actividad compatible*

¿Complejidad?

Un algoritmo greedy

La elección local será tomar, entre las actividades disponibles, la que termina primero. Luego descartamos las que se superponen y repetimos.

```
1: function SELECCIONARACTIVIDADES( $A$ )
2:   ordenar  $A$  por tiempo de finalización creciente
3:    $S \leftarrow \emptyset$ 
4:    $t \leftarrow -\infty$ 
5:   for all  $a \in A$ , en el orden obtenido do
6:     if  $s_a \geq t$  then
7:        $S \leftarrow S \cup \{a\}$ 
8:        $t \leftarrow f_a$ 
9:   return  $S$ 
```

▷ *actividad compatible*

¿Complejidad? $O(n \log n)$

Un algoritmo greedy

La elección local será tomar, entre las actividades disponibles, la que termina primero. Luego descartamos las que se superponen y repetimos.

```
1: function SELECCIONARACTIVIDADES( $A$ )
2:   ordenar  $A$  por tiempo de finalización creciente
3:    $S \leftarrow \emptyset$ 
4:    $t \leftarrow -\infty$ 
5:   for all  $a \in A$ , en el orden obtenido do
6:     if  $s_a \geq t$  then
7:        $S \leftarrow S \cup \{a\}$ 
8:        $t \leftarrow f_a$ 
9:   return  $S$ 
```

▷ *actividad compatible*

¿Complejidad? $O(n \log n)$

Consigna: tómense unos minutos para intentar demostrar que el algoritmo es correcto.

Planteamiento de la Demostración

Queremos demostrar que la solución S devuelta por el algoritmo es **óptima** (maximiza la cantidad de actividades).

Definición de las Soluciones

Sean las actividades de cada conjunto ordenadas por tiempo de finalización creciente:

- Solución Greedy: $S = \{g_1, g_2, \dots, g_k\}$
- Solución Óptima: $O = \{o_1, o_2, \dots, o_m\}$

Planteamiento de la Demostración

Queremos demostrar que la solución S devuelta por el algoritmo es **óptima** (maximiza la cantidad de actividades).

Definición de las Soluciones

Sean las actividades de cada conjunto ordenadas por tiempo de finalización creciente:

- Solución Greedy: $S = \{g_1, g_2, \dots, g_k\}$
- Solución Óptima: $O = \{o_1, o_2, \dots, o_m\}$

Queremos probar por contradicción que $|S| = |O|$, es decir, que $k = m$.

Estrategia: Argumento por Intercambio

Demostraremos que podemos transformar la solución óptima O en nuestra solución greedy S , elemento a elemento, sin disminuir el número total de actividades seleccionadas.

El Paso de Intercambio

Supongamos que S y O coinciden en las primeras r actividades, pero difieren en la siguiente:

$$g_1 = o_1, g_2 = o_2, \dots, g_r = o_r, \quad \text{pero} \quad g_{r+1} \neq o_{r+1}$$

Si difieren en la primera actividad, para lo siguiente pueden pensar en una tarea ficticia que comparten y termina en $t = 0$.

- Como el algoritmo ordena A por tiempo de finalización creciente, la actividad g_{r+1} es la que tiene el **mínimo** f_a disponible que cumple con $s_a \geq f_{g_r}$.
- Dado que $g_r = o_r$, la actividad o_{r+1} de la solución óptima también debe cumplir $s_{o_{r+1}} \geq f_{g_r}$, por lo que era una opción válida para el algoritmo.

El Paso de Intercambio

Supongamos que S y O coinciden en las primeras r actividades, pero difieren en la siguiente:

$$g_1 = o_1, g_2 = o_2, \dots, g_r = o_r, \quad \text{pero} \quad g_{r+1} \neq o_{r+1}$$

Si difieren en la primera actividad, para lo siguiente pueden pensar en una tarea ficticia que comparten y termina en $t = 0$.

- Como el algoritmo ordena A por tiempo de finalización creciente, la actividad g_{r+1} es la que tiene el **mínimo** f_a disponible que cumple con $s_a \geq f_{g_r}$.
- Dado que $g_r = o_r$, la actividad o_{r+1} de la solución óptima también debe cumplir $s_{o_{r+1}} \geq f_{g_r}$, por lo que era una opción válida para el algoritmo.

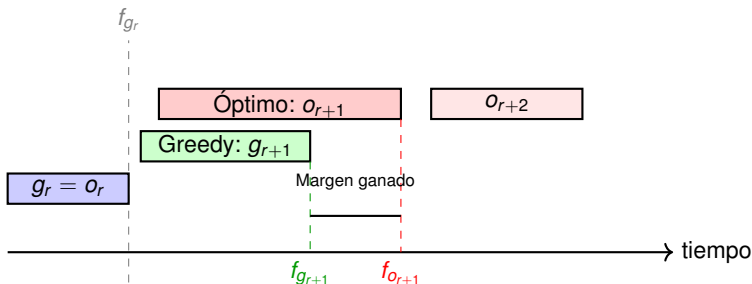
Propiedad Greedy

Por la regla de selección del algoritmo, se garantiza que:

$$f_{g_{r+1}} \leq f_{o_{r+1}}$$

Visualización del Paso de Intercambio

Por hipótesis $P(r)$, la solución óptima actual es $O_r = \{g_1, \dots, g_r, o_{r+1}, o_{r+2}, \dots\}$. Comparamos la opción óptima o_{r+1} con la opción greedy g_{r+1} .



Como el algoritmo elige el fin mínimo, se cumple que $f_{g_{r+1}} \leq f_{o_{r+1}}$.

Validez de la Nueva Solución

Definimos una nueva solución modificada O' reemplazando o_{r+1} por g_{r+1} :

$$O' = (O \setminus \{o_{r+1}\}) \cup \{g_{r+1}\}$$

¿Por qué O' sigue siendo una solución válida?

- **Por la izquierda:** g_{r+1} es compatible con g_r (ya que $g_r = o_r$).
- **Por la derecha:** Como $f_{g_{r+1}} \leq f_{o_{r+1}}$, la actividad greedy termina antes o al mismo tiempo que la óptima. Esto implica que:

$$f_{g_{r+1}} \leq f_{o_{r+1}} \leq s_{o_{r+2}}$$

Por lo tanto, g_{r+1} no se superpone con la siguiente actividad de la secuencia óptima (o_{r+2}).

Como O' no presenta solapamientos y tiene tamaño $|O'| = |O|$, O' **es óptima**.

Conclusión de la demostración

- Aplicando este argumento de forma inductiva, podemos sustituir consecutivamente todos los elementos de la solución óptima hasta que O se transforme en S .

Conclusión

- Greedy puede verse como un caso particular de programación dinámica donde sabemos que alcanza con hacer solo una de las llamadas recursivas.
- Greedy es una estrategia constructiva: en cada paso agrando la solución garantizando factibilidad.
- En este caso, la verificación de factibilidad es trivial y se puede hacer muy eficientemente.
- Además del método de demostración que vimos hay otro que es útil en algunos problemas: *Greedy Stays Ahead*. Sin embargo, el que vimos sirve para todos los algoritmos greedy.

Problema de cambio de monedas

Problema

Tenemos monedas de denominaciones positivas $c_1, \dots, c_r$ y queremos formar un monto entero m . Buscamos usar la menor cantidad posible de monedas.

Consignas

- Tómense unos minutos para escribir una recurrencia de programación dinámica.
- Propongan también un algoritmo greedy para resolver el problema.

Solución por programación dinámica

Sea $F(x)$ la cantidad mínima de monedas para formar el monto x . Si no se puede formar x , $F(x) = +\infty$.

$$F(x) = \begin{cases} 0, & x = 0, \\ 1 + \min_{\substack{1 \leq i \leq r \\ c_i \leq x}} F(x - c_i), & x > 0. \end{cases}$$

La elección greedy para monedas

Una opción natural es tomar siempre la moneda de mayor denominación que no supere el monto que falta.

```
1: function CAMBIOGREEDY( $m, c_1, \dots, c_r$ )
2:   ordenar las denominaciones de mayor a menor
3:    $R \leftarrow m, S \leftarrow \emptyset$ 
4:   for all denominación  $c$ , de mayor a menor do
5:     while  $c \leq R$  do
6:       agregar una moneda  $c$  a  $S$ 
7:        $R \leftarrow R - c$ 
8:   if  $R \neq 0$  then
9:     return “no se puede formar el monto”
10:  return  $S$ 
```

La elección greedy para monedas

Una opción natural es tomar siempre la moneda de mayor denominación que no supere el monto que falta.

```
1: function CAMBIOGREEDY( $m, c_1, \dots, c_r$ )
2:   ordenar las denominaciones de mayor a menor
3:    $R \leftarrow m, S \leftarrow \emptyset$ 
4:   for all denominación  $c$ , de mayor a menor do
5:     while  $c \leq R$  do
6:       agregar una moneda  $c$  a  $S$ 
7:        $R \leftarrow R - c$ 
8:   if  $R \neq 0$  then
9:     return “no se puede formar el monto”
10:  return  $S$ 
```

¿Complejidad?

La elección greedy para monedas

Una opción natural es tomar siempre la moneda de mayor denominación que no supere el monto que falta.

```
1: function CAMBIOGREEDY( $m, c_1, \dots, c_r$ )
2:   ordenar las denominaciones de mayor a menor
3:    $R \leftarrow m, S \leftarrow \emptyset$ 
4:   for all denominación  $c$ , de mayor a menor do
5:     while  $c \leq R$  do
6:       agregar una moneda  $c$  a  $S$ 
7:        $R \leftarrow R - c$ 
8:   if  $R \neq 0$  then
9:     return "no se puede formar el monto"
10:  return  $S$ 
```

¿Complejidad? En peor caso, $O(r \cdot \log r + (m/c_1) \cdot \log r)$ pero se puede implementar en $O(r \cdot \log r)$ agregando $R \text{ div } C$ monedas en cada iteración

La elección greedy para monedas

Una opción natural es tomar siempre la moneda de mayor denominación que no supere el monto que falta.

```
1: function CAMBIOGREEDY( $m, c_1, \dots, c_r$ )
2:   ordenar las denominaciones de mayor a menor
3:    $R \leftarrow m, S \leftarrow \emptyset$ 
4:   for all denominación  $c$ , de mayor a menor do
5:     while  $c \leq R$  do
6:       agregar una moneda  $c$  a  $S$ 
7:        $R \leftarrow R - c$ 
8:   if  $R \neq 0$  then
9:     return “no se puede formar el monto”
10:  return  $S$ 
```

¿Complejidad? En peor caso, $O(r \cdot \log r + (m/c_1) \cdot \log r)$ pero se puede implementar en $O(r \cdot \log r)$ agregando $R \text{ div } C$ monedas en cada iteración

El greedy no funciona siempre

Contraejemplo

Denominaciones 1, 3, 4, monto 6.

- Greedy: $4 + 1 + 1$, usa 3 monedas.
- Óptimo: $3 + 3$, usa 2 monedas.

El greedy no funciona siempre

Contraejemplo

Denominaciones 1, 3, 4, monto 6.

- Greedy: $4 + 1 + 1$, usa 3 monedas.
- Óptimo: $3 + 3$, usa 2 monedas.

¿Bajo qué condiciones sí podemos garantizar que funciona?

Una condición suficiente

Supongamos que las denominaciones están ordenadas

$$1 = c_1 < c_2 < \dots < c_r$$

y que cada denominación divide a la siguiente:

$$c_i \mid c_{i+1} \quad \text{para todo } 1 \leq i < r.$$

Ejemplo

1, 5, 10, 50, 100 cumple la condición: cada denominación divide a la siguiente.

Una condición suficiente

Supongamos que las denominaciones están ordenadas

$$1 = c_1 < c_2 < \dots < c_r$$

y que cada denominación divide a la siguiente:

$$c_i \mid c_{i+1} \quad \text{para todo } 1 \leq i < r.$$

Ejemplo

1, 5, 10, 50, 100 cumple la condición: cada denominación divide a la siguiente.

Consigna: tómense unos minutos para demostrar que el algoritmo greedy es óptimo bajo esta condición.

Demostración para denominaciones divisibles

Sea $q_i = c_{i+1}/c_i$. En una representación óptima, no puede haber q_i o más monedas de denominación c_i : podemos reemplazar q_i monedas de c_i por una moneda de c_{i+1} , reduciendo la cantidad total. Más aún, en una solución óptima, las monedas de denominación c_i suman más que las denominaciones menores.

Llamemos a esto **característica de una solución óptima**.

Sean las monedas de cada conjunto ordenadas decrecientemente por valor:

- Solución Greedy: $S = \{g_1, g_2, \dots, g_t, g_{t+1} \dots, o_k\}$
- Solución Óptima: $O = \{o_1, o_2, \dots, o_t, o_{t+1} \dots, o_m\}$

Queremos probar por contradicción que $|S| = |O|$.

El Paso de Intercambio

Supongamos que S y O coinciden en las primeras r monedas, pero difieren en la siguiente:

$$g_1 = o_1, g_2 = o_2, \dots, g_t = o_t, \quad \text{pero} \quad g_{t+1} \neq o_{t+1}$$

(pueden diferir en la primer moneda)

- $g_{t+1} > o_{t+1}$ por el funcionamiento de greedy. Como están ordenadas, g_{t+1} no aparece en el resto de O .
- Por la **característica de una solución óptima** mencionada, la suma de las monedas restantes en G de denominación g_{t+1} es mayor a la de las restantes en O . Pero eso no es posible dado que tienen que devolver el mismo monto total y hasta t lo hacían.
- Llegamos a un absurdo producto de suponer existía una primera moneda en que diferían.

Demostración alternativa para denominaciones divisibles

Sea $q_i = c_{i+1}/c_i$. En una representación óptima, no puede haber q_i o más monedas de denominación c_i : podemos reemplazar q_i monedas de c_i por una moneda de c_{i+1} , reduciendo la cantidad total.

Demostración alternativa para denominaciones divisibles

Sea $q_i = c_{i+1}/c_i$. En una representación óptima, no puede haber q_i o más monedas de denominación c_i : podemos reemplazar q_i monedas de c_i por una moneda de c_{i+1} , reduciendo la cantidad total.

Aplicamos repetidamente estos reemplazos, empezando por las denominaciones más pequeñas. Obtenemos una representación óptima en la que, para todo $i < r$, la cantidad de monedas c_i es menor que q_i .

- Esos restos son determinados de manera única por las divisiones sucesivas del monto por $c_r, c_{r-1}, \dots, c_1$.
- Son exactamente las cantidades que elige el algoritmo greedy.
- Por lo tanto, greedy produce una representación óptima.

Conclusiones

- No es trivial saber cuando se puede usar greedy.
- La mejora de complejidad es crítica con respecto a programación dinámica.
- La comprobación de factibilidad del greedy es sencilla.

Árbol generador mínimo

Problema

Dado un grafo conexo no dirigido $G = (V, E)$, con peso $w(e)$ en cada arista, queremos hallar un árbol generador de peso mínimo.

Un árbol generador es un subgrafo que:

- conecta todos los vértices;
- no tiene ciclos.

Una definición equivalente: es un grafo sin ciclos, con todos los vértices y exactamente $|V| - 1$ aristas.

Árbol generador mínimo

Problema

Dado un grafo conexo no dirigido $G = (V, E)$, con peso $w(e)$ en cada arista, queremos hallar un árbol generador de peso mínimo.

Un árbol generador es un subgrafo que:

- conecta todos los vértices;
- no tiene ciclos.

Una definición equivalente: es un grafo sin ciclos, con todos los vértices y exactamente $|V| - 1$ aristas.

Consigna: tómense unos minutos para pensar una solución de programación dinámica.

Una programación dinámica exhaustiva

Generalizamos el problema a $BGM(i, A)$, donde buscamos el bosque generador mínimo que extiende a A con i aristas

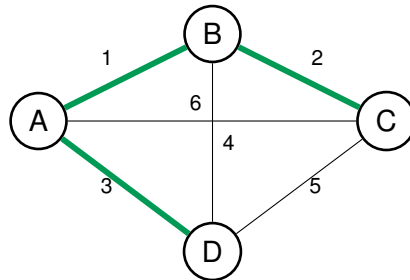
$$DP(i, A) = \min \begin{cases} w(e_i) + DP(i+1, A \cup \{e_i\}) & \text{(Tomar } e_i, \text{ si } A \cup \{e_i\} \text{ es acíclico)} \\ DP(i+1, A) & \text{(No tomar } e_i) \end{cases}$$

es el costo óptimo. Los estados inválidos tienen costo $+\infty$.

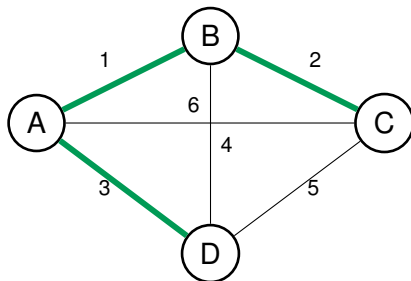
Observación

Es una solución válida, pero muy costosa: hay que considerar 2^m subconjuntos de aristas.

Grafo de ejemplo



Grafo de ejemplo



Consigna: propongan un algoritmo goloso para el árbol generador mínimo.

Algoritmo de Kruskal

Ordenamos las aristas por peso y las agregamos de menor a mayor, siempre que no formen un ciclo.

```
1: function KRUSKAL( $G = (V, E)$ )
2:    $T \leftarrow \emptyset$ 
3:   inicializar una estructura de conjuntos disjuntos con cada vértice
4:   ordenar  $E$  por peso creciente
5:   for all  $e = \{u, v\} \in E$ , en ese orden do
6:     if FIND( $u$ )  $\neq$  FIND( $v$ ) then
7:        $T \leftarrow T \cup \{e\}$ 
8:       UNION( $u, v$ )
9:     if  $|T| = |V| - 1$  then
10:      break
11:   return  $T$ 
```

Algoritmo de Kruskal

Ordenamos las aristas por peso y las agregamos de menor a mayor, siempre que no formen un ciclo.

```
1: function KRUSKAL( $G = (V, E)$ )
2:    $T \leftarrow \emptyset$ 
3:   inicializar una estructura de conjuntos disjuntos con cada vértice
4:   ordenar  $E$  por peso creciente
5:   for all  $e = \{u, v\} \in E$ , en ese orden do
6:     if FIND( $u$ )  $\neq$  FIND( $v$ ) then
7:        $T \leftarrow T \cup \{e\}$ 
8:       UNION( $u, v$ )
9:     if  $|T| = |V| - 1$  then
10:      break
11:   return  $T$ 
```

Consigna: tómense unos minutos para demostrar que funciona.

Kruskal: argumento de intercambio

Sea T el árbol generador mínimo que ya contiene todas las aristas elegidas por Kruskal hasta cierto paso. Consideremos la próxima arista $e = \{u, v\}$ que Kruskal agrega.

- Como e conecta componentes distintas, T no la contiene.
- Al agregar e a O , se forma un ciclo.
- En ese ciclo hay otra arista e' que cruza entre las componentes que e conectaba.
- e' no pudo haber sido elegida antes por Kruskal: si lo hubiera sido, conectaría esas componentes.

Como e fue considerada antes que e' , $w(e) \leq w(e')$.

Kruskal: argumento de intercambio

Sea T el árbol generador mínimo que ya contiene todas las aristas elegidas por Kruskal hasta cierto paso. Consideremos la próxima arista $e = \{u, v\}$ que Kruskal agrega.

- Como e conecta componentes distintas, T no la contiene.
- Al agregar e a O , se forma un ciclo.
- En ese ciclo hay otra arista e' que cruza entre las componentes que e conectaba.
- e' no pudo haber sido elegida antes por Kruskal: si lo hubiera sido, conectaría esas componentes.

Como e fue considerada antes que e' , $w(e) \leq w(e')$.

Al reemplazar e' por e , obtenemos un árbol generador de peso no mayor que sigue conteniendo las elecciones de Kruskal.

Kruskal: conclusión

El intercambio muestra que, después de cada elección de Kruskal, existe un árbol generador mínimo que contiene todas las aristas elegidas hasta el momento.

- El algoritmo agrega aristas solo cuando conectan componentes distintas, así que nunca crea ciclos.
- Termina al tener $|V| - 1$ aristas.
- Un bosque con $|V| - 1$ aristas tiene una sola componente: es un árbol generador.
- Por el argumento de intercambio, ese árbol es mínimo.

Kruskal: complejidad

¿Complejidad?

Kruskal: complejidad

¿Complejidad?

- union y find se pueden implementar en $O(\log n)$ (cota superior holgada).
- $O(m.\log m + m.\log n = m.\log n)$
- Justificación: $\log m \leq \log n^2 = 2.\log n = O(\log n)$

Kruskal: Conclusiones

- La verificación de factibilidad al agrandar es no trivial.
- Eso hace que no se pueda ordenar a priori como pasaba en los otros casos.

Una variante a Kruskal: Prim

Generalizamos el problema a $AIM(i, A)$, donde buscamos el árbol inducido mínimo que extiende a A con i aristas

$$DP(i, A) = \min \begin{cases} w(e_i) + DP(i+1, A \cup \{e_i\}) & \text{(Tomar } e_i, \text{ si } A \cup \{e_i\} \text{ es conexo)} \\ DP(i+1, A) & \text{(No tomar } e_i) \end{cases}$$

es el costo óptimo. Los estados inválidos tienen costo $+\infty$.

Ejercicio

Convertir esta función en un algoritmo con una implementación eficiente y calcular su complejidad.

Ayuda: para que una arista extienda un árbol debe tener un extremo en el mismo y el otro fuera.

Compresión con recuperación perfecta

Usamos codificaciones donde todos los caracteres ocupan la misma cantidad de bits (8 en ASCII, 16 en UTF, etc).

Una forma habitual de comprimir: usar menos bits para los caracteres más frecuentes (y menos para los más frecuentes).

Observación

Si $a = "0"$, $b = "1"$, $c = "00"$, $d = "001"$ y queremos descomprimir "001" tenemos varias posibilidades

- "aab"
- "cb"
- "d"

Compresión con recuperación perfecta

Usamos codificaciones donde todos los caracteres ocupan la misma cantidad de bits (8 en ASCII, 16 en UTF, etc).

Una forma habitual de comprimir: usar menos bits para los caracteres más frecuentes (y menos para los más frecuentes).

Observación

Si $a = "0"$, $b = "1"$, $c = "00"$, $d = "001"$ y queremos descomprimir "001" tenemos varias posibilidades

- "aab"
- "cb"
- "d"

Solución: codigos libres de prefijos.

Codificación de Huffman

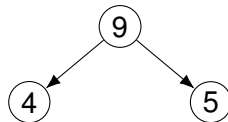
Problema

Tenemos símbolos $a_1, \dots, a_n$, cada uno con frecuencia o peso $w_i > 0$. Queremos asignarles códigos binarios de longitud variable que sean libres de prefijos y minimicen la longitud total ponderada:

$$\sum_{i=1}^n w_i \ell_i.$$

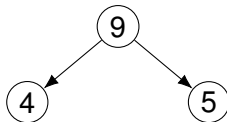
Un código es libre de prefijos si ningún código asignado a un símbolo es prefijo del código de otro. Se representa mediante un árbol binario: los símbolos están en las hojas y ℓ_i es la profundidad de la hoja i .

La elección greedy para Huffman



- Tomar los dos símbolos de menor frecuencia.
- Reemplazarlos por un símbolo combinado, de peso igual a la suma.
- Repetir hasta que quede un único símbolo.

La elección greedy para Huffman



- Tomar los dos símbolos de menor frecuencia.
- Reemplazarlos por un símbolo combinado, de peso igual a la suma.
- Repetir hasta que quede un único símbolo.

```
1: function HUFFMAN( $S$ )
2:   insertar cada símbolo, con su peso, en una cola de prioridad
3:   while la cola tiene más de un elemento do
4:      $x \leftarrow$  extraer el elemento de menor peso
5:      $y \leftarrow$  extraer el elemento de menor peso
6:     insertar un nodo nuevo de peso  $w_x + w_y$ , con hijos  $x, y$ 
7:   return el árbol que queda en la cola
```

Mochila fraccionaria

Problema

Hay n objetos. El objeto i tiene peso $w_i > 0$ y valor $v_i > 0$. La mochila soporta un peso máximo W . Se puede llevar una fracción de cada objeto. Queremos maximizar el valor total.

Mochila fraccionaria

Problema

Hay n objetos. El objeto i tiene peso $w_i > 0$ y valor $v_i > 0$. La mochila soporta un peso máximo W . Se puede llevar una fracción de cada objeto. Queremos maximizar el valor total.

Consigna: propongan una solución greedy. ¿Por qué criterio ordenarían los objetos?

Greedy por valor: no funciona

Consideremos capacidad $W = 10$ y los objetos:

	Objeto 1	Objeto 2	Objeto 3
Peso	10	6	4
Valor	100	90	60

Contraejemplo

Ordenar por valor decreciente elige primero el objeto 1 y obtiene valor 100. En cambio, los objetos 2 y 3 entran juntos y dan valor 150.

Greedy por peso: tampoco funciona

Consideremos capacidad $W = 10$ y los objetos:

$$(w_1, v_1) = (1, 1), \quad (w_2, v_2) = (10, 100).$$

Contraejemplo

Ordenar por peso creciente toma primero el objeto 1; luego solo entran 9/10 del objeto 2, para un valor total de 91. Tomar el objeto 2 entero da valor 100.

Greedy por peso: tampoco funciona

Consideremos capacidad $W = 10$ y los objetos:

$$(w_1, v_1) = (1, 1), \quad (w_2, v_2) = (10, 100).$$

Contraejemplo

Ordenar por peso creciente toma primero el objeto 1; luego solo entran 9/10 del objeto 2, para un valor total de 91. Tomar el objeto 2 entero da valor 100.

¿Qué criterio parece capturar el valor que obtenemos por cada unidad de peso?

Ejercicio: ordenar por valor por gramo

Para cada objeto, definamos su densidad:

$$\rho_i = \frac{v_i}{w_i}.$$

Demostrar que el algoritmo que toma los objetos en orden decreciente de ρ_i , usando una fracción del último si hace falta, es óptimo.

Pista

Si una solución usa algo de un objeto de menor densidad mientras deja fuera peso de uno de mayor densidad, intercambien esas cantidades.

Cierre

- La programación dinámica considera distintas decisiones y guarda subproblemas.
- Un algoritmo greedy toma una única decisión local constructiva en cada paso.
- Para demostrar que greedy funciona, buscamos una propiedad de intercambio.
- A veces chequear factibilidad en cada paso no es trivial.